

DOCUMENTATION TECHNIQUE

Déploiement WireGuard via Ansible sur Windows dans un domaine Active Directory

OPNsense 26.1 | Ansible | WireGuard

Mai 2026

1. Contexte et Infrastructure

Ce document décrit le déploiement automatisé du client WireGuard sur des PCs Windows membres d'un domaine Active Directory, avec création automatique des peers sur OPNsense 26.1 via son API REST.

1.1 Infrastructure

Composant	Valeur
Serveur VPN	OPNsense 26.1.7
IP LAN OPNsense	10.10.0.1
IP Publique OPNsense	82.67.89.83
Instance WireGuard	wg-cuser (wg2, port 51822)
Réseau tunnel VPN	10.150.0.0/24
Réseau serveurs internes	10.0.40.0/24
Réseau LAN clients	10.0.10.0/24
Serveur Ansible	PFR-Ansible (Ubuntu)
Domaine Active Directory	cyna.labo
Contrôleur de domaine (DC)	10.0.40.2

2. Configurer WinRM sur Windows

WinRM est le protocole utilisé par Ansible pour communiquer avec les machines Windows. Il doit être configuré sur chaque PC avant le déploiement.

2.1 Activer WinRM

Exécuter en PowerShell administrateur sur chaque PC (ou via GPO) :

```
winrm quickconfig -y
winrm set winrm/config/service/auth '{@Basic="true"}'
winrm set winrm/config/service '{@AllowUnencrypted="true"}'
netsh advfirewall firewall add rule name="WinRM" protocol=TCP dir=in
localport=5985 action=allow
```

2.2 Vérifier la connectivité depuis Ansible

Depuis le serveur Ansible, tester la connectivité WinRM :

```
ansible windows_clients -i inventory.yml -m win_ping --ask-vault-pass
```

Résultat attendu :

```
CYPO003 | SUCCESS => {"ping": "pong"}
```

2.3 Prérequis sur le control node Ansible

```
pip install ansible pywinrm
ansible-galaxy collection install community.windows ansible.windows
apt install wireguard-tools -y
```

3. Créer la clé API OPNsense

L'API REST d'OPNsense est utilisée pour créer et gérer les peers WireGuard automatiquement.

3.1 Générer les credentials API

- Aller dans System → Access → Users → votre utilisateur
- Descendre jusqu'à API keys → cliquer sur +
- Télécharger le fichier contenant key et secret
- Donner les droits : Services: WireGuard, VPN: WireGuard

3.2 Tester l'accès à l'API

```
OPNS_AUTH="votre_api_key:votre_api_secret"
curl -k -u "$OPNS_AUTH" https://10.10.0.1/api/wireguard/general/get
```

Résultat attendu :

```
{"general":{"enabled":"1"}}
```

3.3 Endpoints API utilisés (OPNsense 26.1)

Important : OPNsense 26.1 intègre WireGuard nativement. Les anciens endpoints du plugin n'existent plus.

Endpoint	Méthode	Description
/api/wireguard/general/get	GET	Vérifier que WireGuard est actif
/api/wireguard/server/getServer/{uuid}	GET	Récupérer config d'une instance
/api/wireguard/client/addClient	POST	Créer un peer
/api/wireguard/client/getClient/{uuid}	GET	Détails d'un peer
/api/wireguard/server/setServer/{uuid}	POST	Lier peers à une instance
/api/wireguard/service/reconfigure	POST	Appliquer la configuration

4. Structure des fichiers Ansible

4.1 Arborescence du projet

```
wireguard-deploy/
├── inventory.yml
├── wireguard_deploy.yml
├── templates/
│   └── wg-cuser.conf.j2
├── group_vars/
│   ├── all/
│   │   └── vault.yml          # Secrets chiffrés
│   └── windows_clients.yml    # Variables communes
└── host_vars/
    ├── CYP0003.yml           # IP tunnel propre à chaque PC
    ├── CYP0004.yml
    └── CYP0005.yml
```

4.2 Inventaire inventory.yml

```
all:
  children:
    windows_clients:
      hosts:
        CYP0003:
          ansible_host: 10.0.10.101
        CYP0004:
          ansible_host: 10.0.10.102
      vars:
        ansible_user: Administrateur
        ansible_password: "{{ vault_win_password }}"
        ansible_connection: winrm
        ansible_winrm_transport: basic
        ansible_port: 5985
```

4.3 Variables communes group_vars/windows_clients.yml

```
wg_instance_name: "wg-cuser"
wg_instance_uuid: "3bd86c39-1dc6-4f59-8f67-13b5fbafe09f"
wg_server_public_key: "tLgVZLSj77RO+C5FyDN6PIr+LLZLxFnHU2e4QC90tkY="
wg_server_endpoint: "82.67.89.83"
wg_server_port: 51822
wg_allowed_ips: "0.0.0.0/0,::/0"
wg_dns: "10.0.40.2,8.8.8.8"
wg_config_path: "C:\\Program Files\\WireGuard\\Data\\Configurations"
wg_tunnel_name: "wg-cuser"
```

4.4 Variables par machine host_vars/CYP0003.yml

```
wg_client_ip: "10.150.0.2"
```

Chaque machine doit avoir une IP unique dans le réseau tunnel 10.150.0.0/24.

4.5 Secrets chiffrés group_vars/all/vault.yml

```
vault_opnsense_api_key: "votre_api_key"
```

```
vault_opnsense_api_secret: "votre_api_secret"  
vault_win_password: "MotDePasseAdmin"
```

Chiffrer le vault avec la commande :

```
ansible-vault encrypt group_vars/all/vault.yml
```

5. Template de configuration WireGuard

Fichier : templates/wg-cuser.conf.j2

```
[Interface]
PrivateKey = {{ client_private_key }}
Address = {{ wg_client_ip }}/32
DNS = {{ wg_dns }}

[Peer]
PublicKey = {{ wg_server_public_key }}
Endpoint = {{ wg_server_endpoint }}:{{ wg_server_port }}
AllowedIPs = {{ wg_allowed_ips }}
PersistentKeepalive = 25
```

Important : AllowedIPs = 0.0.0.0/0,::/0 est obligatoire pour inclure IPv6, sinon la route par défaut n'est pas correctement installée sur Windows.

6. Playbook principal wireguard_deploy.yml

6.1 Gestion des clés

Les clés privées sont générées une seule fois et sauvegardées sur le PC dans :

```
C:\Program Files\WireGuard\Data\Configurations\privatekey
```

À chaque run Ansible, si le fichier existe, la clé est réutilisée — évitant de recréer un nouveau peer.

```
- name: Vérifier si une clé privée existe déjà sur le PC
  win_stat:
    path: "{{ wg_config_path }}\privatekey"
    register: existing_key

- name: Générer la clé privée si absente
  command: wg genkey
  register: wg_private_key_new
  delegate_to: localhost
  when: not existing_key.stat.exists
```

6.2 Récupération de l'instance OPNsense

L'UUID de l'instance et les peers existants sont récupérés avant toute modification pour ne pas écraser la configuration existante.

```
- name: Récupérer la config actuelle de l'instance
  uri:
    url: "{{ opnsense_host }}/api/wireguard/server/getServer/{{
wg_instance_uuid }}"
    method: GET
    user: "{{ opnsense_api_key }}"
    password: "{{ opnsense_api_secret }}"
    force_basic_auth: true
    validate_certs: false
    register: current_instance
    delegate_to: localhost
```

6.3 Création du peer sur OPNsense

Points importants lors de la création d'un peer :

- Le champ serveraddress doit être vide sinon OPNsense tente de contacter le client
- Il faut envoyer tous les UUIDs des peers existants + le nouveau lors du setServer
- Un reconfigure est nécessaire pour appliquer la configuration

```
- name: Créer le peer sur OPNsense si absent
  uri:
    url: "{{ opnsense_host }}/api/wireguard/client/addClient"
    method: POST
    body:
      client:
        enabled: "1"
        name: "{{ inventory_hostname }}"
        pubkey: "{{ wg_public_key.stdout }}"
        tunneladdress: "{{ wg_client_ip }}/32"
        serveraddress: ""
        serverport: ""
```

```
endpoint: ""
keepalive: "25"
```

6.4 Déploiement sur Windows

WireGuard sur Windows chiffre automatiquement la config en .conf.dpapi. Le script doit gérer les deux cas :

```
$dpapi = "C:\...\wg-cuser.conf.dpapi"
$conf = "C:\...\wg-cuser.conf"
if (Test-Path $dpapi) {
    wireguard.exe /installtunnelservice $dpapi
} elseif (Test-Path $conf) {
    wireguard.exe /installtunnelservice $conf
}
```

Avant de modifier la config, il faut arrêter le tunnel ET le WireGuard Manager car il verrouille les fichiers :

```
wireguard.exe /uninstalltunnelservice wg-cuser
Stop-Service -Name "WireGuardManager" -Force
# modifier la config
Start-Service -Name "WireGuardManager"
wireguard.exe /installtunnelservice wg-cuser.conf.dpapi
```

7. Configuration Firewall OPNsense

7.1 Règles WAN

Autoriser les connexions WireGuard entrantes sur le port UDP 51822 :

Action	Protocole	Source	Destination	Port
Pass	IPv4 UDP	*	WAN adresse	51822

7.2 Règles interface wg2 (wg-cuser)

Règles pour l'authentification au domaine Active Directory et l'accès Internet :

Protocole	Source	Destination	Port	Description
IPv4 UDP	wgcuser net	10.0.40.2	53	DNS UDP
IPv4 TCP	wgcuser net	10.0.40.2	53	DNS TCP
IPv4 TCP/UDP	wgcuser net	10.0.40.2	88	Kerberos
IPv4 TCP	wgcuser net	10.0.40.2	135	RPC
IPv4 TCP	wgcuser net	10.0.40.2	139	NetBIOS
IPv4 TCP/UDP	wgcuser net	10.0.40.2	389	LDAP
IPv4 TCP	wgcuser net	10.0.40.2	445	SMB / Sysvol
IPv4 TCP	wgcuser net	10.0.40.2	49152-65535	RPC dynamique
IPv4 *	wgcuser net	! RFC1918	*	Accès Internet

8. Lancer le déploiement

8.1 Test sur un seul PC

```
ansible-playbook wireguard_deploy.yml -i inventory.yml \  
--limit CYP0003 --ask-vault-pass
```

8.2 Mode verbose pour déboguer

```
ansible-playbook wireguard_deploy.yml -i inventory.yml \  
--limit CYP0003 --ask-vault-pass -v
```

8.3 Déploiement sur tout le parc

Avant de lancer sur tous les PCs, s'assurer que chaque machine a son IP tunnel définie dans `host_vars` :

```
ansible-playbook wireguard_deploy.yml -i inventory.yml --ask-vault-pass
```

9. Commandes de vérification

9.1 Vérifier l'état du VPN sur OPNsense

Via SSH sur OPNsense :

```
wg show wg2
```

Résultat attendu avec un tunnel établi :

```
peer: RzLWVSfqOFtcf52mI0g7aC+tbNPPbgZ1auDxWhY0hWU=
  endpoint: 192.168.1.44:57374
  allowed ips: 10.150.0.2/32
  latest handshake: 1 minute, 46 seconds ago
  transfer: 43.39 KiB received, 40.56 KiB sent
```

9.2 Vérifier l'état du VPN sur Windows

```
# Services WireGuard
Get-Service | Where-Object {$_.Name -like "*WireGuard*"}

# Statistiques de l'interface
Get-NetAdapterStatistics -Name "wg-cuser"

# Routes installées
Get-NetRoute -DestinationPrefix "0.0.0.0/0"

# Config déployée
type "C:\Program Files\WireGuard\Data\Configurations\wg-cuser.conf"
```

9.3 Tester la connectivité

```
# Depuis le PC client via VPN
ping 10.10.0.1      # OPNsense LAN
ping 10.0.40.2     # DC / DNS
ping 10.0.40.2     # Serveurs internes
```

10. Points importants et problèmes rencontrés

10.1 API OPNsense 26.1 — Endpoints modifiés

OPNsense 26.1 intègre WireGuard nativement. Les anciens endpoints du plugin n'existent plus :

Ancienne API (plugin)	Nouvelle API (natif 26.x)
/api/wireguard/server/searchServers	N'existe plus
/api/wireguard/client/searchClients	N'existe plus
/api/wireguard/server/getServer/{uuid}	Fonctionne
/api/wireguard/client/addClient	Fonctionne
/api/wireguard/server/setServer/{uuid}	Fonctionne

10.2 Champ serveraddress vide obligatoire

Lors de la création d'un peer client, les champs serveraddress et serverport doivent être vides. Sinon OPNsense tente de contacter le client et la connexion échoue.

10.3 AllowedIPs IPv6 obligatoire sur Windows

Sur Windows, il faut inclure `::/0` dans AllowedIPs pour que la route par défaut soit correctement installée. Sans cela, WireGuard n'installe pas la route `0.0.0.0/0` et aucun trafic ne passe par le tunnel.

```
AllowedIPs = 0.0.0.0/0,::/0
```

10.4 Fichier conf chiffré par Windows (DPAPI)

WireGuard sur Windows chiffre automatiquement la configuration en `.conf.dpapi`. Le script d'activation doit gérer les deux cas (`conf` et `conf.dpapi`).

10.5 Conserver les peers existants lors du setServer

Lors de l'ajout d'un peer, il faut envoyer tous les UUIDs des peers existants + le nouveau, sinon les anciens sont désélectionnés de l'instance.

```
peers: "uuid1,uuid2,uuid3,nouveau_uuid"
```

10.6 NAT Hairpin

Le VPN ne fonctionne pas si le PC client est sur le même réseau qu'OPNsense (même IP publique). Il faut tester depuis un réseau avec une IP publique différente. C'est un comportement normal du NAT.

10.7 DNS pour l'authentification domaine

Configurer deux serveurs DNS dans la config WireGuard : le DC en premier, puis un DNS public en fallback. Cela évite le deadlock où le tunnel ne peut pas s'établir car le DNS interne est inaccessible avant la connexion VPN.

```
DNS = 10.0.40.2,8.8.8.8
```